🖥️ CampusLands Asset Manager — Control Interno de Equipos

Sistema de consola desarrollado en Python + SQLite para gestionar el préstamo de computadores en las instalaciones de Cajasan (CampusLands). Permite controlar asignaciones, estados físicos de equipos y estados de usuarios con trazabilidad completa.

📁 Estructura del Proyecto

```
gestionEquipos/
│
├── core/
│   ├── __init__.py
│   ├── database.py     # Inicialización del esquema SQLite
│   └── manager.py      # Lógica de negocio (CampusManager)
│
├── main.py             # Interfaz de usuario (menú de consola)
└── campuslands.db      # Base de datos generada automáticamente
```

⚙️ Requisitos

- Python 3.x
- Librería estándar `sqlite3` (incluida en Python)
- No requiere dependencias externas

🚀 Cómo ejecutar

```
python main.py
```

La base de datos `campuslands.db` se crea automáticamente al iniciar el programa.

🗃️ Esquema de Base de Datos

`campers`

Campo	Tipo	Descripción
id	INTEGER	Clave primaria (ingresada manualmente)
nombre	TEXT	Nombre completo del camper
estado	TEXT	`Activo` | `Bloqueado`

`computadores`

Campo	Tipo	Descripción
serial	TEXT	Clave primaria
marca	TEXT	Marca del equipo
estado_fisico	TEXT	`Excelente` | `Dañado`
disponibilidad	TEXT	`Disponible` | `Ocupado` | `Bloqueado`

`asignaciones`

Campo	Tipo	Descripción
id_asignacion	INTEGER	Clave primaria autoincremental
id_camper	INTEGER	FK → campers.id
serial_pc	TEXT	FK → computadores.serial
cubiculo	INTEGER	Número de cubículo asignado
fecha_inicio	TIMESTAMP	Fecha/hora de inicio de la asignación
fecha_fin	TIMESTAMP	Fecha/hora de devolución
activo	INTEGER	`1` = activo, `0` = histórico

📋 Funcionalidades del Menú

Opción	Acción
1 — Registro	Registrar nuevo Camper o PC
2 — Asignar	Vincular un PC disponible a un Camper
3 — Retornar	Liberar un equipo y registrar fecha de devolución
4 — Admin	Bloquear/desbloquear Camper · Reportar PC dañado
5 — Reportes	Ver asignaciones activas · Ver PCs disponibles
6 — Salir	Cerrar el sistema

🔒 Reglas de Negocio

- Un Camper **bloqueado** no puede recibir equipos.
- Un PC **dañado** queda bloqueado automáticamente para nuevas asignaciones.
- Un Camper **no puede tener más de un PC** activo simultáneamente.
- Al reportar daño técnico sobre un PC en uso, el sistema **cierra la asignación automáticamente**.
- Todas las operaciones críticas se ejecutan en **transacciones atómicas**.

🤖 Trazabilidad de Ingeniería IA — Flujo Completo de Prompts

A continuación se documenta el flujo real de prompts utilizados entre conversaciones, siguiendo la cadena:

Product Owner → Developer → QA → Developer → Product Owner → Developer → QA

💬 CONVERSACIÓN 1 — Product Owner

🔁 Prompt 1 — Arranque del proyecto

Prompt enviado:

Actúa como un Product Owner profesional. Temática: control interno de los computadores que usan los campers de CampusLands en las instalaciones de Cajasan. Antes de empezar, ¿qué información necesitas para poder desarrollar esto sin problemas, siguiendo el paso a paso de un PO que sabe lo que hace?
NO QUIERO QUE GENERES TEXTO DE MANERA INNECESARIA, haz las preguntas que tengas que hacer.

Resultado obtenido: El agente asumió el rol de PO y, antes de proponer cualquier solución, formuló 4 bloques de preguntas estructuradas: definición del problema y objetivos, usuarios y roles, alcance técnico y restricciones, y priorización del MVP. No generó código ni propuestas hasta tener contexto suficiente.

🔁 Prompt 2 — Respuesta del Stakeholder

Prompt enviado:

1. Gestión desordenada → gestión eficaz, saber quién lo usó, si está disponible, poder bloquear usuarios y que los PCs dañados se puedan bloquear.
2. Estudiantes y administrador(es) — no se registra nada.
3. ID del camper, cubículo asignado, estado físico del equipo — escritorio, Python, menú consola — no hay integración con terceros.
4. La gestión de usuarios.

Resultado obtenido: El PO procesó las respuestas y entregó la propuesta estructurada: explicación del sistema como "portero digital", flujo operativo completo con tres módulos (autenticación/bloqueo, inventario dinámico, trazabilidad) y un ejemplo concreto del flujo en consola.

🔁 Prompt 3 — Confirmación del Stack

Prompt enviado:

Usaremos Python conectado con SQLite.

Resultado obtenido: El PO formalizó la arquitectura técnica y describió el modelo CRUD sobre tres entidades (campers, computadores, asignaciones). Explicó el flujo lógico de validaciones en Python antes de cada transacción y las ventajas de SQLite para el contexto del proyecto.

🔁 Prompt 4 — Solicitud del Prompt para el Developer

Prompt enviado:

No, tú serás el PO, así que no te preocupes por código. Lo que quiero es que ahora me generes un prompt para usarlo en otra conversación, donde expliques lo que queremos y pidas el código como lo necesitamos, que asuma rol de programador senior con experiencia en el campo requerido.

Resultado obtenido: El PO generó el briefing técnico completo para entregar al Developer: rol asignado, stack, 4 user stories, 3 reglas de negocio críticas y entregables esperados. Este prompt fue el puente formal entre la conversación de negocio y la conversación de desarrollo.

💬 CONVERSACIÓN 2 — Senior Developer (Iteración 1)

🔁 Prompt 5 — Kick-off con el Developer

Prompt enviado:

Rol: Senior Python Developer con amplia experiencia en arquitectura de software y gestión de bases de datos relacionales ligeras.

Contexto: MVP para control interno de equipos en CampusLands.
Stack: Python 3.x + sqlite3 + menú de consola.

Requerimientos funcionales:
- Gestión de campers con bloqueo.
- Gestión de PCs con estado físico.
- Control de asignación por cubículo.
- Trazabilidad histórica.

Reglas críticas:
- Un Camper no puede tener más de un PC activo simultáneamente.
- Un PC dañado u ocupado no puede asignarse.
- Validar existencia antes de cualquier transacción.

Entregables: init_db(), clase CampusManager, menú while-True con try-except robusto.

Resultado obtenido: El Developer entregó los tres archivos del MVP funcional: `database.py` con el esquema de tres tablas y llaves foráneas, `manager.py` con la clase `CampusManager` con métodos CRUD y validaciones de negocio, y `main.py` con el menú interactivo. El sistema era funcional pero contenía vulnerabilidades de atomicidad que serían detectadas en la etapa de QA.

🔁 Prompt 6 — Consulta sobre estructura de archivos

Prompt enviado:

¿Cómo debo llamar cada archivo?

Resultado obtenido: El Developer definió la estructura modular `core/` con `__init__.py`, `database.py` y `manager.py`, más `main.py` en la raíz. Justificó cada nombre siguiendo principios de separación de responsabilidades y entregó el ejemplo de imports correcto.

🔁 Prompt 7 — Solicitud del Prompt para el QA

Prompt enviado:

Quiero que generes un prompt para pedirle a otra conversación que actúe como tester QA y que revise el código.

Resultado obtenido: El Developer generó el briefing de QA con rol asignado, contexto del proyecto, 4 ejes de validación (edge cases, integridad SQLite, validación de entradas, reglas de negocio) y entregables esperados. Este prompt fue el puente formal entre desarrollo y calidad.

💬 CONVERSACIÓN 3 — QA Engineer (Primera Revisión)

🔁 Prompt 8 — Kick-off con el QA

Prompt enviado:

Rol: Senior QA Automation Engineer especializado en pruebas de integración y validación de reglas de negocio en Python.

Contexto: Sistema de consola para controlar préstamo de computadores.
Python 3 + SQLite. Debe impedir que usuarios bloqueados o PCs dañados
entren en el flujo de asignación.

Valida:
- Casos de borde: asignación a usuario inexistente, PC ya ocupado.
- Integridad SQLite: transacciones, conexiones, llaves foráneas.
- Validación de entradas: robustez del try-except.
- Reglas de negocio: no doble PC, PC dañado inutilizable,
  trazabilidad al liberar.

Entregables: reporte de bugs, sugerencias de mejora, script de pruebas.

[Código adjunto: database.py, manager.py, main.py]

Resultado obtenido: El QA entregó un reporte técnico con bugs clasificados por categoría y severidad:

#	Severidad	Hallazgo
1	Alta	Atomicidad rota en `asignar_equipo`: dos conexiones distintas, riesgo de PC "fantasma"
2	Alta	`PRAGMA foreign_keys` no activado por conexión, solo en `init_db`
3	Media	Seriales duplicados fallan silenciosamente sin mensaje amigable
4	Media	Doble liberación de PC sin error ni validación
5	Baja	Entradas vacías aceptadas en nombre y marca
6	Baja	`try-except` genérico mezcla errores de BD con errores de input

Adicionalmente entregó un script de `unittest` con casos de prueba para las reglas de negocio principales.

🔄 CONVERSACIÓN 2 — Developer aplica correcciones del QA

🔁 Prompt 9 — Entrega del reporte QA al Developer

Prompt enviado:

Esto me contestó el tester, ¿qué aconsejas y cómo lo vamos a llevar a cabo?

[Reporte completo del QA adjunto]

Resultado obtenido: El Developer propuso un plan de acción en tres pasos y entregó el código refactorizado. Los cambios clave fueron: método `_get_connection()` centralizado que activa `PRAGMA foreign_keys = ON` en cada conexión, toda la lógica de `asignar_equipo` unificada en un solo bloque `with` (transacción atómica), validación de campos vacíos en registros, `limpiar_pantalla()` con `os.system`, y mensajes de éxito/error retornados desde todos los métodos del manager.

🔄 CONVERSACIÓN 1 — Product Owner define la segunda iteración

🔁 Prompt 10 — Solicitud de siguiente paso al PO

Prompt enviado:

¿Qué aconsejas a continuación? (mejoras tales como listar equipos o campers, etc. ¿Qué paso deberíamos seguir?) Escribe el prompt para pasárselo al desarrollador directamente.

[Código actualizado post-refactorización adjunto]

Resultado obtenido: El PO evaluó que el sistema era funcional pero "estaba a ciegas": el administrador no podía ver qué PCs estaban libres ni quiénes tenían equipos sin intentar una transacción fallida. Definió el backlog de la segunda iteración enfocado en visibilidad operativa y generó el prompt formal para el Developer con nuevos requerimientos de consultas, gestión de estados y mejoras de UX.

🔄 CONVERSACIÓN 2 — Developer implementa la segunda iteración

🔁 Prompt 11 — Kick-off Segunda Iteración

Prompt enviado:

Asunto: Implementación de Módulos de Consulta y Gestión de Estados (Iteración 2)

Objetivos:
- Listar PCs disponibles: serial y marca, estado Disponible y físico Excelente.
- Ver asignaciones activas: JOIN con nombre del Camper, Serial, Cubículo y fecha.
- Bloquear/desbloquear Camper sin borrar el registro.
- Reportar daño técnico: cambiar estado a 'Dañado' y liberar asignación activa automáticamente registrando fecha_fin y activo=0.
- Mostrar PCs disponibles antes de pedir el serial en Asignar Equipo (UX preventiva).
- Tabla visual en consola con columnas alineadas.

Regla adicional: si el PC está en uso al momento del daño, cerrar la asignación en la misma transacción.

Entregables: manager.py actualizado + main.py actualizado.

Resultado obtenido: El Developer entregó `manager.py` con cuatro métodos nuevos: `obtener_pcs_disponibles()`, `obtener_asignaciones_activas()` con JOIN, `gestionar_estado_camper()` con validación de `rowcount`, y `reportar_daño_tecnico()` con liberación en cascada dentro de una transacción atómica. El `main.py` fue reorganizado en categorías semánticas (REGISTRO / ASIGNAR / RETORNAR / ADMIN / REPORTES) y se incorporó `mostrar_tabla()` para visualización alineada en consola.

🔄 CONVERSACIÓN 3 — QA valida la segunda iteración

🔁 Prompt 12 — Segunda revisión al QA

Prompt enviado:

[Código completo de la segunda iteración adjunto: manager.py con los nuevos métodos y main.py con el menú reorganizado]

Estos fueron los cambios que realizó el desarrollador.

Resultado obtenido: El QA confirmó que los bugs críticos de atomicidad e integridad fueron resueltos correctamente. Identificó observaciones menores pendientes para futuras iteraciones: validación de dominio en `gestionar_estado_camper` (acepta cualquier string), posible

desalineación de tabla con nombres largos, y ausencia de feedback diferenciado al reportar daño en PC ya dañado. Dio el visto bueno a las reglas de negocio principales.

🧪 Script de Pruebas de Reglas de Negocio

Generado por el agente QA para verificar las reglas críticas del sistema:

```
import unittest
import os
from core.manager import CampusManager
from core.database import init_db

class TestCampusLogic(unittest.TestCase):

    def setUp(self):
        self.db_test = 'test_campus.db'
        self.manager = CampusManager(self.db_test)
        init_db()

    def tearDown(self):
        if os.path.exists(self.db_test):
            os.remove(self.db_test)

    def test_no_doble_asignacion(self):
        """Un Camper no puede tener 2 PCs activos simultáneamente."""
        self.manager.registrar_camper(1, "Test User")
        self.manager.registrar_pc("ABC", "Dell")
        self.manager.registrar_pc("XYZ", "HP")
        self.manager.asignar_equipo(1, "ABC", 101)
        res = self.manager.asignar_equipo(1, "XYZ", 102)
        self.assertEqual(res, "Error: El Camper ya tiene un equipo asignado.")

    def test_pc_danado_no_asignable(self):
        """Un PC dañado no puede entrar en el flujo de asignación."""
        self.manager.registrar_camper(2, "User 2")
        self.manager.registrar_pc("BAD", "Lenovo")
        self.manager.reportar_daño_tecnico("BAD")
        res = self.manager.asignar_equipo(2, "BAD", 105)
        self.assertIn("Error", res)

if __name__ == '__main__':
    unittest.main()
```

📌 Backlog — Observaciones Pendientes (identificadas por QA)

- [ ] Validar que `nuevo_estado` en `gestionar_estado_camper` solo acepte `['Activo', 'Bloqueado']`.
- [ ] Agregar columna `observaciones` en `asignaciones` para notas al liberar equipos.
- [ ] Resumen de inventario: total de PCs, ocupados, disponibles y dañados.
- [ ] Historial por equipo: todos los campers que han usado un PC específico.
- [ ] Mejorar alineación de tabla para nombres largos.
- [ ] Exportación de reportes a CSV/JSON.

👥 Roles IA en el Proceso de Desarrollo

Rol	Conversación	Responsabilidad
Product Owner	Conversación 1	Elicitación de requerimientos, backlog, prompts hacia el equipo
Senior Developer	Conversación 2	Implementación MVP, refactorización, decisiones de arquitectura
QA Engineer	Conversación 3	Análisis de bugs, pruebas de reglas de negocio, validación de iteraciones

---

Proyecto desarrollado como ejercicio de Ingeniería IA aplicada al ciclo de vida de software — CampusLands / Cajasan.